Especificación

Diseño e Implementación de un Lenguaje

5 de Abril de 2026

1. Equipo 1
2. Repositorio 1
3. Dominio 2
4. Construcciones 2
5. Casos de prueba 3
6. Ejemplo 4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Carlos Amador	Vallejo Tapia	65454	cvallejotapia@itba.edu.ar
Bautista Santiago	Simone	65516	bsimone@itba.edu.ar
Jaime	Nazar Anchorena	65064	jnazaranchorena@itba.edu.ar
Lucas	Desmery	65663	ldesmery@itba.edu.ar

2. Repositorio

La solución será versionada en: https://github.com/AmadorCVT/ATLC_TP_1C2026

3. Dominio

Desarrollar un lenguaje que permita realizar operaciones con autómatas finitos, ya sea deterministas, no deterministas o no deterministas lambda. Dicho lenguaje permitirá evaluar si una palabra es aceptada o no, determinar si dos autómatas son equivalentes, minimizar autómatas finitos deterministas, y realizar la conversión entre los distintos tipos.

La correcta implementación del mismo permitirá reducir el trabajo mecánico que requiere operar con autómatas, principalmente los algoritmos de conversión y minimización. Además, permitirá mostrar, mediante la práctica, el comportamiento de los autómatas finitos y cómo se comparan entre sí.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

2.1 Definición de Autómatas

P-1) Se podrá crear un autómata de tipo DFA, NFA o LNFA especificando alfabeto, estados, estado inicial y estados de aceptación.
P-2) Se podrán declarar transiciones con sintaxis q0 -> símbolo : destino o conjuntos q0 -> símbolo : {q1, q2}
P-3) Las transiciones lambda se declaran con la palabra reservada lambda como símbolo.
P-4) Se podrán crear autómatas vacíos (sin ninguna transición) para completar luego.

2.2 Modificación de Autómatas

P-5) Se podrán agregar o eliminar transiciones de un autómata ya definido usando la sintaxis de bloque NombreAutomata { transitions { ... } }
P-6) Se podrán agregar nuevos estados y redefinir el conjunto de aceptación.

2.3 Ejecución y Testeo

P-7) Se podrá testear una cadena sobre un autómata con test <A> with "cadena". Devuelve accept o reject.
P-8) Se soporta la cadena vacía como entrada ("").
P-9) La ejecución sobre LNFA calcula automáticamente la clausura-λ en cada paso.

2.4 Conversiones

P-10) Se podrá convertir un autómata a otro tipo con convert <A> to <tipo> as <nombre>.
P-11) Conversiones soportadas:
NFA → DFA (subconjuntos)
LNFA → NFA (eliminación de λ)
LNFA → DFA (directa)
DFA → NFA (trivial, agrega no-determinismo vacío)

2.5 Visualización e Inspección
P-12) Se podrá imprimir la definición de un autómata con print <A>.
P-13) Se podrá mostrar la tabla de transición con show table of <A>.
P-14) Se podrá mostrar la clausura-λ de un estado con show closure(<q>) in <A>.
P-15) Se podrá mostrar las transiciones de un autómata con show transitions of <A>.

2.6 Tipos de Datos y Control
P-16) Las variables pueden ser de tipo Automata, String, Bool, State, Symbol.
P-17) Se proveerán estructuras de control: if-then-else, for, while.
P-18) Se proveerán operaciones lógicas: AND, OR, NOT.
P-19) Se proveerán operadores relacionales: < > = ≠ ≤ ≥.

2.7 Prestaciones Opcionales
OP-1) Minimización de DFA (algoritmo de Hopcroft).
OP-2) Verificar equivalencia entre dos autómatas con equivalent <A1> <A2>.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

(I). Un programa que crea un DFA y testea múltiples cadenas.
(II). Un programa que crea un NFA, lo convierte a DFA y testea la equivalencia de resultados.
(III). Un programa que crea un LNFA, calcula clausuras y lo convierte a NFA.
(IV). Un programa que modifica un autómata existente y re-testea cadenas.
(V). Un programa que muestra la tabla de transición de un DFA construido por conversión.
(VI). Un programa que verifica si la cadena vacía es aceptada por distintos autómatas.
(VII). Un programa que convierte un LNFA directamente a DFA y compara resultados.
(VIII). Un programa que usa estructuras de control para testear una lista de cadenas.
(IX). Un programa que imprime y muestra transiciones de un autómata antes y después de editarlo.
(X). Un programa completo que define tres autómatas de distintos tipos y los convierte entre sí.

Además, los siguientes casos de prueba de **rechazo**:

(I). Un programa que declara un DFA sin estado inicial.
(II). Un programa que declara una transición con un símbolo no perteneciente al alfabeto.
(III). Un programa que declara un DFA con una transición no determinista (destino múltiple).
(IV). Un programa que declara un LNFA con transición lambda pero lo tipea como DFA.
(V). Un programa que convierte a un tipo inválido (ej.: convert A to LNFA as B desde un DFA sin lambdas).

6. Ejemplo

6.1 Creación de un DFA — Paridad de Ceros

```
// Crear autómata DFA
automaton ParidadCeros : DFA {
    alphabet     = {0, 1}
    states       = {q0, q1}
    start        = q0
    accept       = {q0}
    transitions {
        q0 -> 0 : q1
        q0 -> 1 : q0
        q1 -> 0 : q0
        q1 -> 1 : q1
    }
};
```

6.2 Modificación de un Autómata Existente

```
// Editar transiciones existentes
ParidadCeros {
    transitions {
        q0 -> 1 : q1    // Sobreescribe la transición q0->1
    }
};
```

6.3 Testeo de Autómatas

```
test ParidadCeros with "0011";
test ParidadCeros with "010";
test ParidadCeros with "";
```

6.4 Autómata LNFA con Transiciones Lambda

```
automaton LambdaEjemplo : LNFA {
    alphabet = {a, b}
    states   = {q0, q1, q2}
    start    = q0
    accept   = {q2}
    transitions {
        q0 -> lambda : {q1}    // transición ε
        q1 -> a : {q1}
        q1 -> b : {q2}
    }
};
```

6.5 Autómata NFA — Contiene "ab"

```
automaton TieneAB : NFA {
    alphabet = {a, b}
    states   = {q0, q1, q2}
    start    = q0
    accept   = {q2}
    transitions {
        q0 -> a : {q0, q1}
        q0 -> b : {q0}
        q1 -> b : {q2}
        q2 -> a : {q2}
        q2 -> b : {q2}
    }
};
```

6.6 Conversiones entre Tipos

```
// NFA -> DFA (construcción de subconjuntos)
convert TieneAB to DFA as TieneAB_Det;

// LNFA -> DFA (elimina lambda y determiniza)
convert LambdaEjemplo to DFA as LambdaEjemplo_Det;

// LNFA -> NFA (solo elimina transiciones lambda)
convert LambdaEjemplo to NFA as LambdaEjemplo_SinLambda;
```

6.7 Visualización e Inspección

```
// Imprimir definición completa
print TieneAB;

// Ver tabla de transición
show transitions of TieneAB;
show table of TieneAB_Det;

// Calcular clausura-lambda del estado q0 en el LNFA
show closure(q0) in LambdaEjemplo;
```